

Name: Benedicto, Ruel Jr A.	Date Performed: 1/15/2026
Course/Section: CPE212 - CPE31S1	Date Submitted: 1/15/2026
Instructor: Engr. Robin Valenzuela	Semester and SY: 2nd sem 2025-2026
Activity 5: Consolidating Playbook plays	
1. Objectives: 1.1 Use when command in playbook for different OS distributions 1.2 Apply refactoring techniques in cleaning up the playbook codes	
2. Discussion: We are going to look at a way that we can differentiate a playbook by a host in terms of which distribution the host is running. It's very common in most Linux shops to run multiple distributions, for example, Ubuntu shop or Debian shop and you need a different distribution for a one off-case or perhaps you want to run plays only on certain distributions. It is a best practice in ansible when you are working in a collaborative environment to use the command git pull. git pull is a Git command used to update the local version of a repository from a remote. By default, git pull does two things. Updates the current local working branch (currently checked out branch) and updates the remote-tracking branches for all other branches. git pull essentially pulls down any changes that may have happened since the last time you worked on the repository. Requirement: In this activity, you will need to create a CentOS VM. Likewise, you need to activate the second adapter to a host-only adapter after the installation. Take note of the IP address of the CentOS VM. Make sure to use the command ssh-copy-id to copy the public key to CentOS. Verify if you can successfully SSH to CentOS VM.	
Task 1: Use when command for different distributions	
1. In the local machine, make sure you are in the local repository directory (CPE232_yourname). Issue the command git pull. When prompted, enter the correct passphrase or password. Describe what happened when you issued this command. Did something happen? Why? 2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): ansible-playbook --ask-become-pass install_apache.yml. After	

executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum."

3. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
      when: ansible_distribution == "Ubuntu"

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
      when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

- The playbook ran on all hosts but executed tasks only on Ubuntu systems. It updated the APT package index, installed the Apache2 web server, and added PHP support by installing the Apache PHP module. All actions were performed with administrative privileges, resulting in a PHP-enabled Apache setup on the machine. The playbook executed only on one server because I only open server1 Ubuntu to download.

The screenshot shows a Visual Studio Code editor with a file explorer on the left displaying a project named 'CPE212_31S1'. The files listed are 'ansible.cfg', 'install_apache.yml', 'inventory.ini', 'playbook.yml', and 'README.md'. The main editor window shows the 'install_apache.yml' file with the following content:

```
1 ---
2 - hosts: all
3   become: true
4   tasks:
5
6     - name: update repository index
7       apt:
8         update_cache: yes
9       when: ansible_distribution == "Ubuntu"
10
11    - name: install apache2 package
12      apt:
13        name: apache2
14      when: ansible_distribution == "Ubuntu"
```

Below the editor, the 'TERMINAL' tab is active, showing the command 'ansible-playbook --ask-become-pass inst all apache.yml' and its output:

```
PLAY RECAP *****
****
192.168.56.103 : ok=0    changed=0    unreachable=1    failed=0
               skipped=0    rescued=0    ignored=0
192.168.56.104 : ok=4    changed=1    unreachable=0    failed=0
               skipped=0    rescued=0    ignored=0
```

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

- name: update repository index
apt:
 update_cache: yes
 when: ansible_distribution in ["Debian", "Ubuntu"]

Note: This will work also if you try. Notice the changes are highlighted.

4. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"

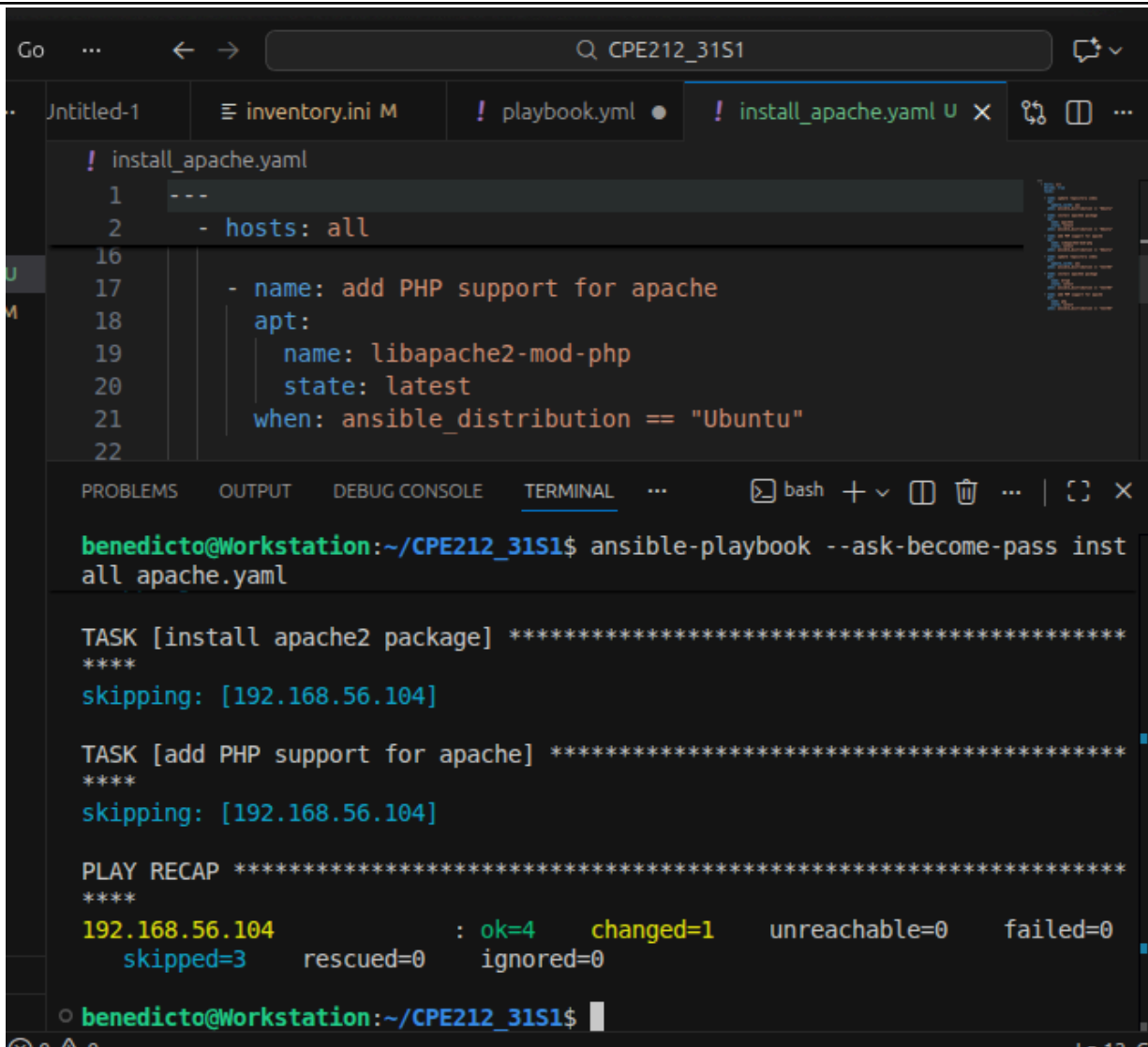
    - name: update repository index
      dnf:
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install apache2 package
      dnf:
        name: httpd
        state: latest
        when: ansible_distribution == "CentOS"

    - name: add PHP support for apache
      dnf:
        name: php
        state: latest
        when: ansible_distribution == "CentOS"
```

Make sure to save and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.



The screenshot shows a VS Code editor with a file named `install_apache.yaml` open. The file contains an Ansible playbook with the following content:

```
1 ---
2 - hosts: all
16
17   - name: add PHP support for apache
18     apt:
19       name: libapache2-mod-php
20       state: latest
21       when: ansible_distribution == "Ubuntu"
22
```

Below the editor, the terminal window shows the output of the command `ansible-playbook --ask-become-pass inst all apache.yaml`. The output indicates that the tasks were skipped for the host `192.168.56.104`.

```
benedicto@Workstation:~/CPE212_31S1$ ansible-playbook --ask-become-pass inst
all apache.yaml

TASK [install apache2 package] *****
****
skipping: [192.168.56.104]

TASK [add PHP support for apache] *****
****
skipping: [192.168.56.104]

PLAY RECAP *****
****
192.168.56.104      : ok=4    changed=1    unreachable=0    failed=0
                    skipped=3    rescued=0    ignored=0

benedicto@Workstation:~/CPE212_31S1$
```

- dfgdsg

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the `httpd` service or the Apache HTTP server in CentOS is not yet active. Thus, you need to activate it first.

5.1 To activate, go to the CentOS VM terminal and enter the following:

`systemctl status httpd`

The result of this command tells you that the service is inactive.

5.2 Issue the following command to start the service:

`sudo systemctl start httpd`

(When prompted, enter the sudo password)

sudo firewall-cmd --add-port=80/tcp

(The result should be a success)

5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)

Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

1. Edit the playbook *install_apache.yml*. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

```

---
- hosts: all
  become: true
  tasks:

    - name: update repository index Ubuntu
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"

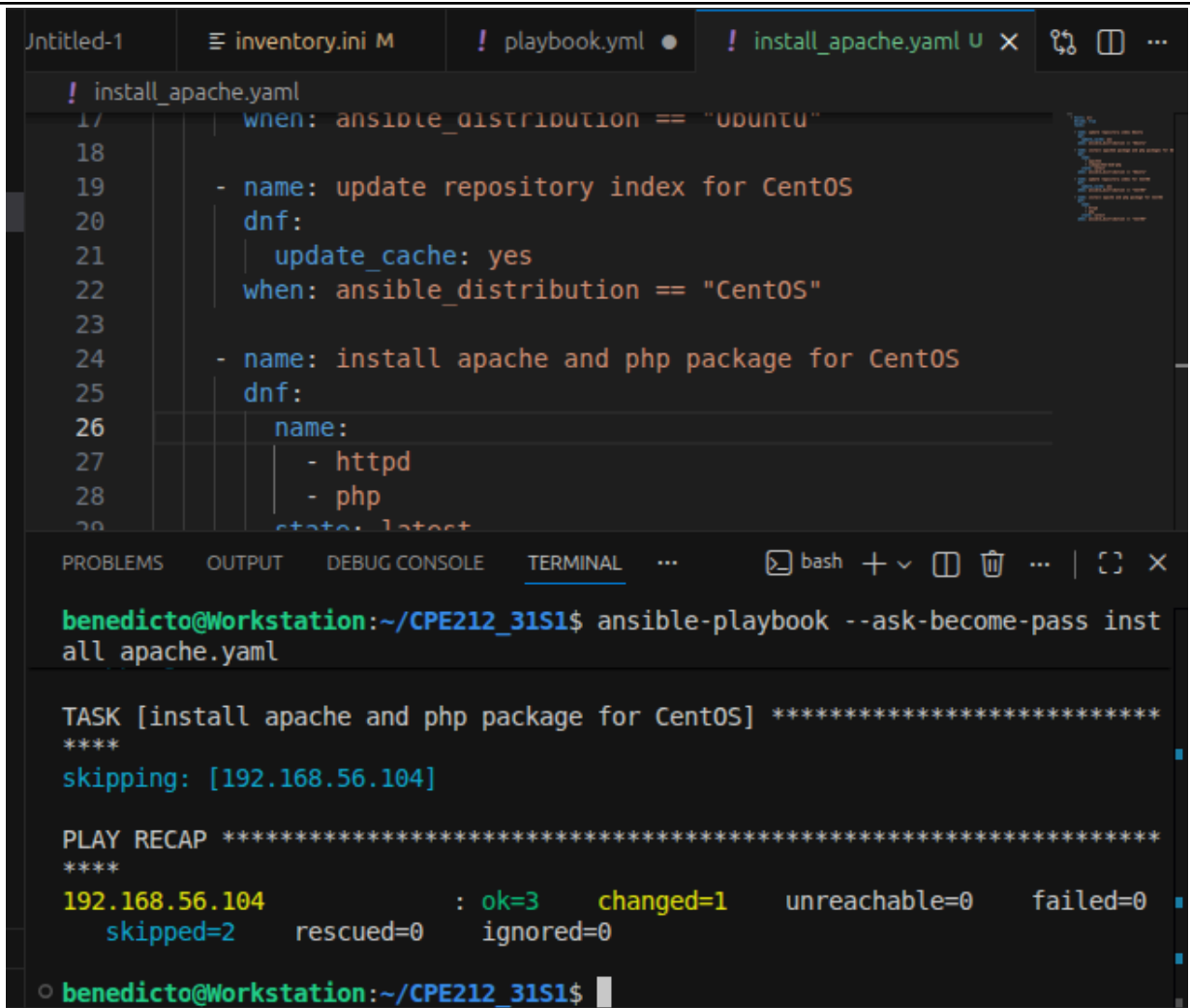
    - name: update repository index for CentOS
      dnf:
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install apache and php packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"

```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.



The screenshot shows an IDE with a file explorer at the top containing 'inventory.ini', 'playbook.yml', and 'install_apache.yml'. The main editor displays the 'install_apache.yml' playbook with the following content:

```
! install_apache.yml
17   when: ansible_distribution == "Ubuntu"
18
19   - name: update repository index for CentOS
20     dnf:
21       update_cache: yes
22     when: ansible_distribution == "CentOS"
23
24   - name: install apache and php package for CentOS
25     dnf:
26       name:
27         - httpd
28         - php
29       state: latest
```

Below the editor is a terminal window with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', and 'TERMINAL'. The terminal shows the command `ansible-playbook --ask-become-pass inst all apache.yml` being executed. The output indicates that the task '[install apache and php package for CentOS]' is skipped for the host 192.168.56.104. A 'PLAY RECAP' section shows that 2 hosts were skipped, 1 was changed, and 0 were unreachable or failed.

```
benedicto@Workstation:~/CPE212_31S1$ ansible-playbook --ask-become-pass inst
all apache.yml

TASK [install apache and php package for CentOS] *****
***
skipping: [192.168.56.104]

PLAY RECAP *****
***
192.168.56.104      : ok=3    changed=1    unreachable=0    failed=0
                    skipped=2    rescued=0    ignored=0

o benedicto@Workstation:~/CPE212_31S1$
```

2. Edit the playbook *install_apache.yml* again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidate everything in just 2 plays. This can be done by removing the update repository play and putting the command *update_cache: yes* below the command *state: latest*. See below for reference:


```
---
- hosts: all
  become: true
  tasks:

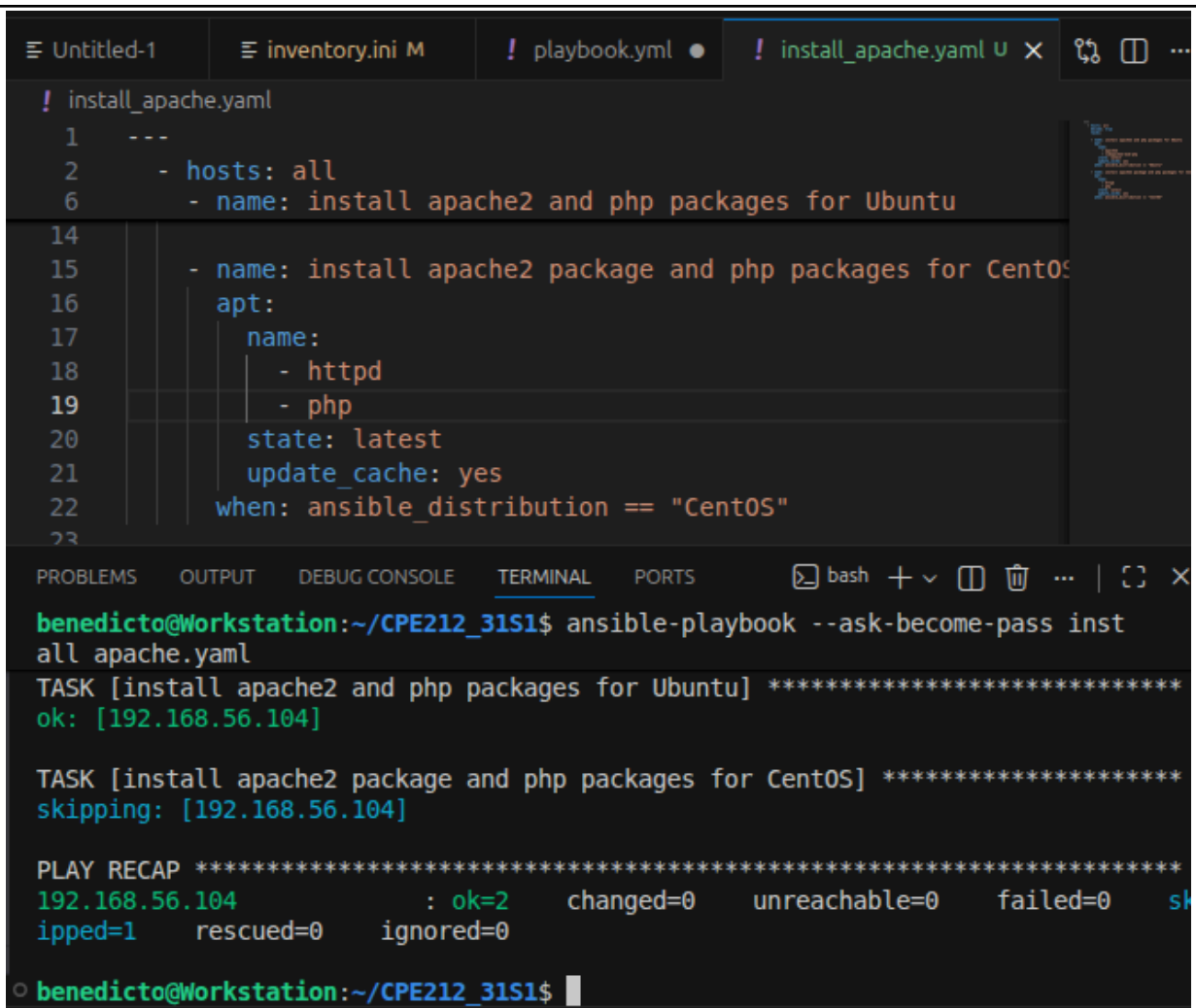
    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        update_cache: yes
      when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

- I noticed in my terminal output that my playbook successfully configured the web server on the remote machine at 192.168.56.101. Since the machine was running Ubuntu, the automation correctly ran the tasks for that OS while skipping over the CentOS steps entirely.



The screenshot shows an IDE with a dark theme. At the top, there are tabs for 'Untitled-1', 'inventory.ini M', 'playbook.yml', and 'install_apache.yml U'. The 'install_apache.yml' tab is active, displaying the following Ansible playbook:

```
! install_apache.yml
1  ---
2  - hosts: all
6    - name: install apache2 and php packages for Ubuntu
14
15    - name: install apache2 package and php packages for CentOS
16      apt:
17        name:
18          - httpd
19          - php
20        state: latest
21        update_cache: yes
22        when: ansible_distribution == "CentOS"
23
```

Below the editor, the 'TERMINAL' tab is active, showing the execution of the playbook. The terminal output is as follows:

```
benedicto@Workstation:~/CPE212_31S1$ ansible-playbook --ask-become-pass inst
all apache.yml
TASK [install apache2 and php packages for Ubuntu] *****
ok: [192.168.56.104]

TASK [install apache2 package and php packages for CentOS] *****
skipping: [192.168.56.104]

PLAY RECAP *****
192.168.56.104      : ok=2    changed=0    unreachable=0    failed=0    sk
ipped=1    rescued=0    ignored=0

benedicto@Workstation:~/CPE212_31S1$
```

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the `apache_package` and `php_package` are variables. The names are arbitrary, which means we can choose different names. We also take out the line `when: ansible_distribution`. Edit the playbook *install_apache.yml* again and make sure to follow the below image. Make sure to save the file and exit.

```

---
- hosts: all
  become: true
  tasks:

    - name: install apache and php
      apt:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest
        update_cache: yes

```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

The screenshot shows an IDE with a file named `install_apache.yml` open. The file contains the following Ansible playbook:

```

1  ---
2  - hosts: all
3
4
5
6  - name: install apache and php
7    apt:
8      name:
9        - "{{ apache_package }}"
10       - "{{ php_package }}"
11      state: latest
12      update_cache: yes
13
14
15

```

Below the editor, the terminal window shows the command `ansible-playbook --ask-become-pass install_apache.yml` being executed. The output indicates an error: `undefined variable. The error was: 'apache_package' is undefined. 'apache_package' is undefined`. The error message suggests that the variable `apache_package` is not defined in the environment or the playbook's `vars` section.

The terminal also shows a `PLAY RECAP` section:

```

PLAY RECAP *****
192.168.56.104 : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0

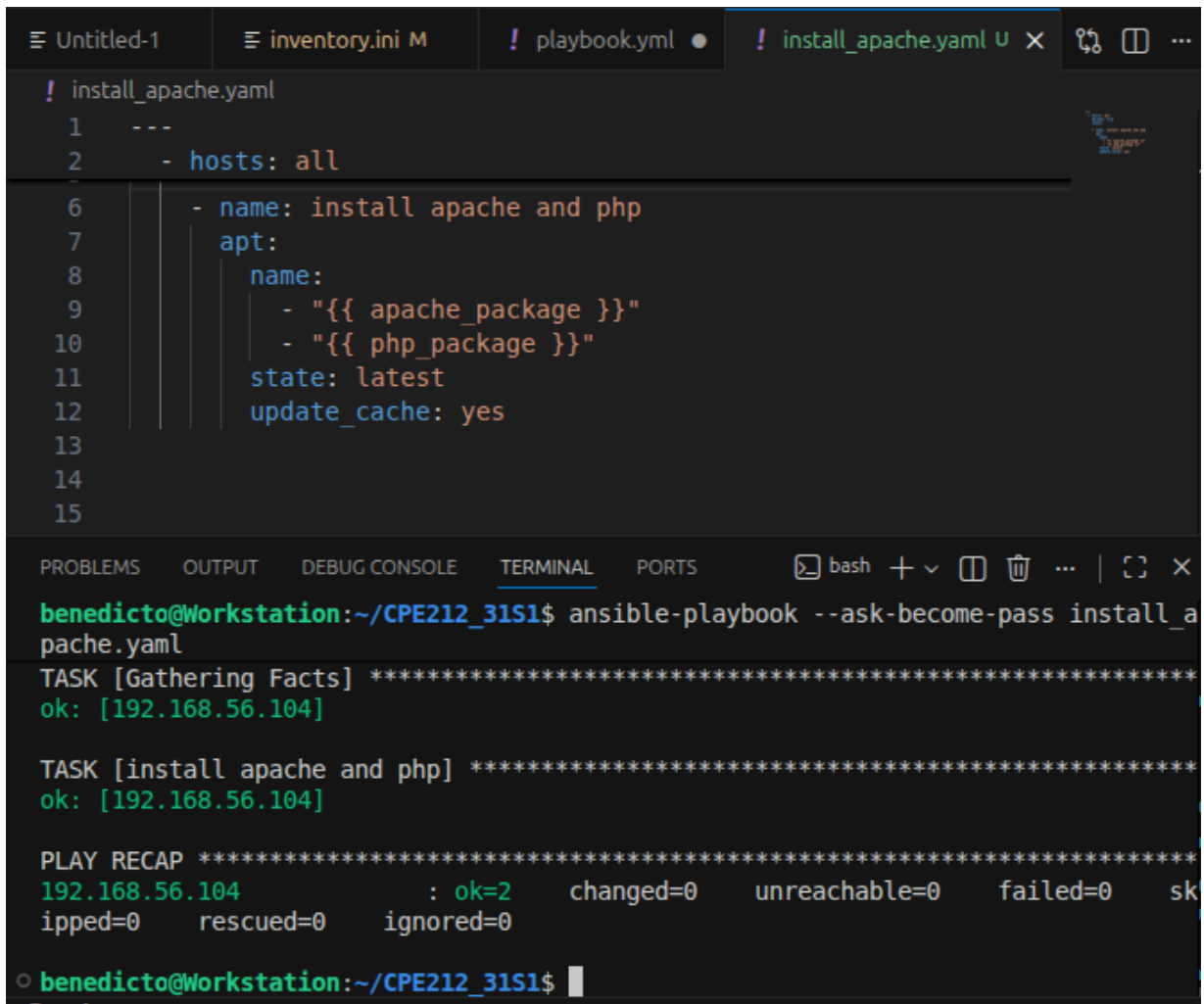
```

The terminal window is titled `bash` and shows the prompt `benedito@Workstation:~/CPE212_31S1$`.

4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the *inventory* file and follow the below configuration:

```
192.168.56.120 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.121 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.122 apache_package=httpd php_package=php
```

Make sure to save the *inventory* file and exit.



The screenshot shows a code editor with the following content in `install_apache.yml`:

```
! install_apache.yml
1 ---
2 - hosts: all
3
4
5
6 - name: install apache and php
7   apt:
8     name:
9       - "{{ apache_package }}"
10      - "{{ php_package }}"
11     state: latest
12     update_cache: yes
13
14
15
```

Below the editor is a terminal window showing the execution of the playbook:

```
benedito@Workstation:~/CPE212_3151$ ansible-playbook --ask-become-pass install_apache.yml
TASK [Gathering Facts] *****
ok: [192.168.56.104]

TASK [install apache and php] *****
ok: [192.168.56.104]

PLAY RECAP *****
192.168.56.104      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

benedito@Workstation:~/CPE212_3151$
```

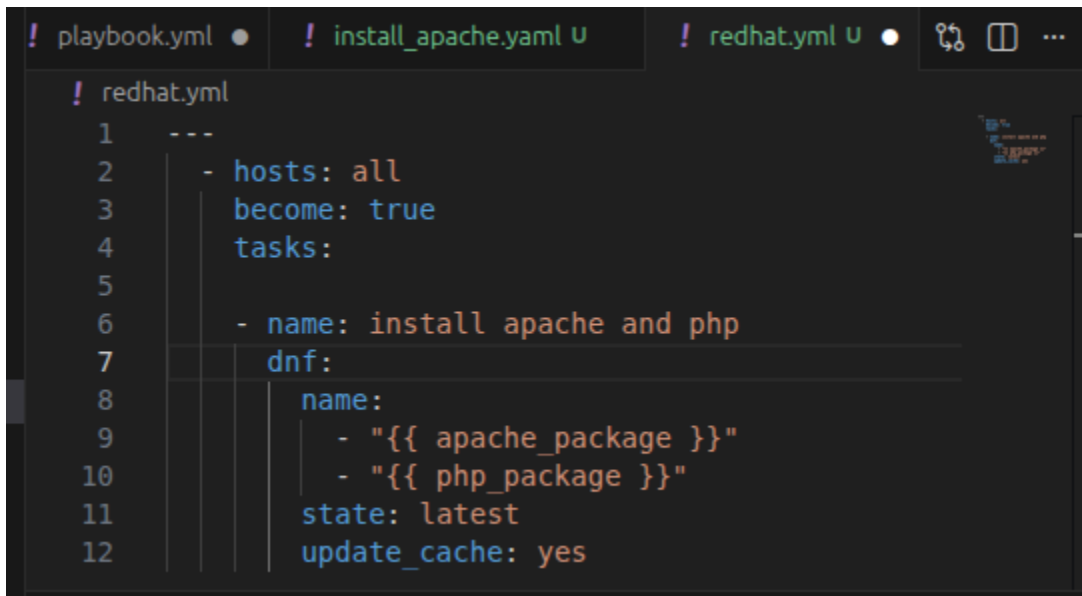
Finally, we still have one more thing to change in our *install_apache.yml* file. In task 2.3, you may notice that the package is assign as *apt*, which will not run in CentOS. Replace the *apt* with *package*. Package is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use *apt*, and for CentOS it will automatically use *dnf*. Make sure to save the file and exit. For more details about the ansible package, you may refer to this [documentation](#):

[ansible.builtin.package – Generic OS package manager — Ansible Documentation](#)

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

Supplementary Activity:

1. Create a playbook that could do the previous tasks in Red Hat OS.
- **This code will run when CentOS is downloaded**



```
! playbook.yml ● ! install_apache.yml U ! redhat.yml U ● 🔍 📄 ...
! redhat.yml
1 ---
2 - hosts: all
3   become: true
4   tasks:
5
6     - name: install apache and php
7       dnf:
8         name:
9           - "{{ apache_package }}"
10          - "{{ php_package }}"
11         state: latest
12         update_cache: yes
```

Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?
 - **The when command is used to run a task only if a specific condition is met, such as verifying whether a server is running Linux or Windows. This helps prevent playbook failures by ensuring that tasks, like software installation, are only executed on compatible systems.**
2. When do we use the “when” command in the playbook?
 - **Refactoring makes automation cleaner and easier to manage by breaking messy files into simple, reusable parts, reducing errors and speeding up updates.**

<https://github.com/benedictocyan/cpe212->

